

OpenClaw Research for mindX Integration

Prepared: 2026-05-12 **Context:** Research into OpenClaw architecture, source patterns, and ecosystem for the purpose of identifying methods and components that improve mindX (mindx.pythai.net), in conjunction with the AgenticPlace × BANKON DAIO deployment.

Executive Summary

OpenClaw — formerly Clawdbot/Moltbot, built by Peter Steinberger around the "Molty" space-lobster mascot — is the dominant open-source self-hosted AI agent runtime as of mid-2026. It carries 370,000+ GitHub stars, 13,729+ skills in its ClawHub registry, and 9.17 trillion all-time tokens served via OpenRouter. It is also the platform that operationalized the SKILL.md AgentSkills format which Anthropic itself adopted (the same convention seen at `/mnt/skills/public/<name>/SKILL.md` in this runtime).

For mindX the integration thesis is **not** "run on OpenClaw" but rather "selectively absorb the patterns mindX currently lacks while preserving the Soul-Mind-Hands / BDI / MASTERMIND architecture and the on-chain DAIO advantage that OpenClaw structurally cannot match." The five highest-leverage transfers are:

1. The SKILL.md / openclaw.plugin.json manifest format and ClawHub-style registry pattern
2. The pluggable Context Engine pattern (v2026.3.7+, lossless-claw flagship)
3. The hybrid vector+BM25 (70/30) Active Memory sub-agent pattern over SQLite + sqlite-vec
4. The plugin manifest-first validation model with capability typing
5. **OpenClaw-RL** — directly applicable to mindXtrain/automindXtrain v3.6 targeting Qwen3.5/3.6 for aGLM derivatives

The 0G OpenClaw track at ETHGlobal OpenAgents (concluded 2026-05-03) explicitly rewarded modular agent-brain libraries with swappable memory layers and self-evolving frameworks on 0G Storage — a profile mindX already structurally satisfies.

1. Architectural Decomposition

1.1 Gateway / Runtime Separation

OpenClaw is a hub-and-spoke system. A central Node.js WebSocket Gateway holds messaging sessions (one WhatsApp session per host, one Telegram bot connection, etc.) and acts as the only process maintaining channel state. The embedded `pi-mono` Agent Runtime runs the iterative

tool-using loop end-to-end: assembling context from session history and memory, invoking the model, executing tool calls, and persisting state. The Gateway and Runtime communicate over a documented wire protocol.

The architectural principle: the LLM provides intelligence; OpenClaw provides the operating system. Conversation history, tool execution, session state, and orchestration logic stay on the operator's infrastructure even when model API calls go to Anthropic/OpenAI/etc.

Map to mindX: mindx.pythai.net is structurally the gateway+runtime. AgenticPlace is the marketplace surface. Agents are sessions. The piece worth borrowing is the **normalized message envelope across channels** — one mindX agent serving Telegram, Discord, the web UI, and on-chain message origins simultaneously through one runtime. The Gateway's channel plugin pattern (`ChannelPlugin` interface, registered with the Gateway, `createChatChannelPlugin` providing session mapping, message chunking, and mention policy resolution) is the right abstraction to copy.

1.2 SKILL.md Format and Skill Resolution

This is the single most adoptable pattern. Skills are folders containing a `SKILL.md` with YAML frontmatter declaring `name`, `description`, and runtime requirements. Skills are loaded as **metadata only** (name + description + file path) into the system prompt; the model reads the full SKILL.md content on demand when it judges the skill relevant. This is precisely what Anthropic adopted for its own skill system.

```
yaml
---
name: my-skill
description: Does a thing with an API.
metadata:
  openclaw:
    requires:
      env: [MY_API_KEY]
      bins: [curl]
    primaryEnv: MY_API_KEY
---
# Skill instructions in markdown
```

Skill resolution order (highest precedence first): workspace skills (user-owned) → managed overrides in `~/.openclaw/skills` → bundled defaults. Higher precedence wins on name conflict. Configured roots support one level of grouping (e.g. `skills/<group>/<skill>/SKILL.md`).

Map to mindX: Publish `SOUL.md`, `MIND.md`, `HANDS.md` companions using the same convention so mindX skills are installable into OpenClaw and vice versa. This is the cheapest way to inherit

the ClawHub long tail (5,400+ categorized skills, ~13.7K total) without inheriting the ClawHub security posture.

Security caveat: A 2026 Lakera audit of 4,310 ClawHub skills documented significant prompt-injection vectors because SKILL.md content is injected directly into the system prompt. A Koi Security audit found 341 malicious entries. mindX must layer manifest validation + static SKILL.md analysis + runtime sandboxing before opening any marketplace.

1.3 Plugin System (openclaw.plugin.json)

Native plugins use a strict manifest-first validation model. The Gateway validates `openclaw.plugin.json` via JSON Schema **before** executing any plugin JavaScript. Manifest fields include `configSchema`, `providers`, `channels`, `providerAuthEnvVars`, `openclaw.compat.pluginApi`, and `openclaw.build.openclawVersion`. Plugins are classified by registration shape: plain-capability, hybrid-capability, hook-only, or non-capability.

Capabilities that plugins can register include channels, model providers, tools, memory backends, context engines, speech, realtime voice, media understanding, image generation, video generation, web fetch, and web search. The Plugin SDK is partitioned into small subpaths (`openclaw/plugin-sdk/core`, `openclaw/plugin-sdk/memory-host-core`, etc.) imported individually rather than as a monolithic entry point.

Hooks documented in production plugins (e.g. memory-lancedb-pro):

- `before_prompt_build` (current; replaces deprecated `before_agent_start`)
- `agent_end` (auto-capture on session close)
- `before_tool_call` / `after_tool_call`
- `agent:bootstrap`, `command:new`, `command:reset`

Map to mindX: This maps almost exactly to BDI control points. MASTERMIND should expose a typed-hook surface so external developers can extend orchestration without forking. The manifest-first validation pattern is the right pre-mainnet safety net for AgenticPlace agent loading.

1.4 Context Engine (v2026.3.7+, March 2026)

OpenClaw introduced pluggable context engines in v2026.3.7, with `lossless-claw` as the flagship community plugin using DAG-based lossless context compression. The engine controls:

- `ingest({ sessionId, message, isHeartbeat })` — store/index incoming messages
- `assemble({ sessionId, messages, tokenBudget, availableTools, citationsMode })` — return token-budgeted message set for the next model run
- `compact({ sessionId, force })` — summarize older history (called on `/compact` or token-limit pressure)

Engines also have optional sub-agent lifecycle hooks (`prepareSubAgent`), etc.) for parent/child session forking with isolated or shared transcript modes.

Map to mindX: This is the same control surface mindX's P-O-D-A reasoning loop applies to context. The lossless-claw DAG approach is worth studying as an alternative or complement to the existing 768-dim consciousness-vector encoding for cross-session belief persistence. The "agent drift after extended sessions" failure mode that motivated lossless-claw is exactly the failure ataraxia is designed to mitigate; cross-pollination is natural.

1.5 Memory System (Active Memory Sub-Agent)

OpenClaw uses an **Active Memory sub-agent** — a blocking sub-agent that runs before the main agent loop, performs hybrid BM25+vector search, and injects relevant "beliefs" into the system prompt. The default weighting is 70% vector / 30% BM25, combined via **union** (not intersection) so a chunk that scores high on vector but zero on keywords still surfaces. The formula is explicit and trivially configurable per user:

```
finalScore = vectorWeight × vectorScore + textWeight × textScore
```

Storage: SQLite with the `sqlite-vec` extension at `~/.openclaw/memory/{agentId}.sqlite`. BM25 uses FTS5. Embedding provider auto-selection priority: Local (`node-llama-cpp` with auto-downloaded GGUF) → OpenAI → Gemini → BM25-only fallback. `candidateMultiplier: 4` fetches extras before fusion. If FTS5 cannot be created, the system falls back to vector-only without hard failure.

The **memory-lancedb-pro** plugin (CortexReach) offers a production-grade upgrade path: LanceDB backend with RRF fusion + Jina cross-encoder rerank + recency boost + importance weight + length normalization + time decay + hard-min-score filter + noise filter + MMR diversity. It supports multi-scope access control: `global`, `agent:<id>`, `custom:<name>`, `project:<id>`, `user:<id>`. The multi-scope pattern is exactly what AgenticPlace needs for multi-tenant memory isolation.

The Milvus team also extracted OpenClaw's memory subsystem and open-sourced it as `memsearch`.

Map to mindX: The BDI Belief layer maps directly. The vector terminology "belief" already appears in OpenClaw's docs (`buildActiveMemoryPromptSection` / "active memory prompt sections"). mindX should formalize its Belief layer with the same hybrid 70/30 contract, expose a swap point for `sqlite-vec` ↔ LanceDB ↔ external (Qdrant, which is already in the mindX Knowledge Catalogue CQRS projection layer), and adopt the multi-scope access-control pattern.

1.6 Tool Execution

Plugin tool factories are cached against an effective request context that includes runtime config, workspace, agent/session IDs, sandbox policy, browser settings, delivery context, requester

identity, and ownership state. Slow factories ($\geq 1s$ individual, $\geq 5s$ aggregate) are surfaced as warnings. The trace logging path (`(openclaw config set logging.level trace)`) surfaces per-plugin factory timing for diagnosis.

A known bug (#56208) — plugin tools registered during `(register())` are unavailable in sub-agent sessions because `(createOpenClawTools())` runs before `(loadOpenClawPlugins())` completes for the sub-agent context — illustrates the maturity tax of dynamic-registration designs. mindX should prefer **static tool declaration in the manifest** with dynamic enrichment (params, descriptions) at register time. This matches the pattern already used by the Anthropic skill system in this very runtime.

1.7 Session Model

Sessions are identified by structured keys (e.g. `(slack:C12345:T67890)`) that map channel + account + thread to state. Transcripts are persisted as JSONL files in the agent's workspace. Session lifecycle includes auto-compaction when token budgets are pressed (`(applyPiAutoCompactionGuard)`) and malformed-file repair (`(repairSessionFileIfNeeded)`). The `(dmScope)` config controls whether all DMs share one session (`(main)`) or each gets its own — relevant for continuity across devices.

Map to mindX: JSONL session transcripts are the right primitive for OG Storage Log archival. The session-key structure should accommodate on-chain origins (`(ethereum:0x...)`, `(algorand:appid:address)`) alongside conventional channels.

2. OpenClaw-RL (Highest-Priority Finding)

Released 2026-02-26 by Gen-Verse (Yinjie Wang, Xuyang Chen, Xiaolong Jin, Mengdi Wang, Ling Yang) at <https://github.com/Gen-Verse/OpenClaw-RL>, **OpenClaw-RL** is a fully asynchronous reinforcement learning framework that turns every agent interaction's **next-state signal** — the user reply, tool output, terminal/GUI state change that follows each action — into a live online training signal.

The thesis: next-state signals are universal across personal conversations, terminal executions, GUI interactions, SWE tasks, and tool-call traces. They are not separate training problems; they are all interactions that can train the same policy in the same loop.

Infrastructure

Four decoupled components on the asynchronous slime framework:

1. **Environment server** — collects samples from Personal Agents (conversational, on-device) and General Agents (terminal, GUI, SWE, tool-call, cloud-hosted)
2. **PRM / Judge** — process reward model / judge for reward computation
3. **Megatron** — policy training

4. SGLang — policy serving with graceful weight updates

The framework supports training with any agentic framework, integrates LoRA training, and has been deployed on Tinker and Fireworks AI. Track 2 explicitly supports Qwen3.5. The team integrated SDFT and SDPO methods on 2026-03-03 and welcomes new method integrations.

Direct Alignment with mindXtrain v3.6

The mindXtrain / automindXtrain v3.6 framework targets Qwen3.5/3.6 as the primary base for aGLM derivatives. OpenClaw-RL is a turnkey upstream:

1. **Adopt slime + SGLang + Megatron as the mindXtrain training substrate** rather than building from scratch
2. **Use "next-state signal" as the universal training-data primitive** across all mindX agent surfaces. Hands-layer tool outputs, Soul-layer reflections, and Mind-layer decisions all become uniform training samples
3. **Process Reward Model (PRM)** is already in production use here for process rewards in general-agent settings — directly applicable to the BDI desire/intention reward shaping
4. **Upstream integration path:** the OpenClaw-RL repo explicitly welcomes new methods. Contributing the mindX-specific reward schemas (e.g. ataraxia clarity-quality, consciousness-vector alignment) is the cheapest way to get aGLM derivatives noticed by the right research community

GLM-5.1 specialist track deferred for hardware: `openclaw-combine` (`openclaw-combine/run_qwen3_4b_openclaw_topk_select.sh`) shows the team's exact bash invocation pattern for Qwen3-4B; the same harness will run for a quantized aGLM training prototype on commodity hardware.

3. Self-Improving Patterns

3.1 The self-improving-agent Skill

The `self-improving-agent` skill (Peter Skoett, widely forked) captures learnings, errors, and corrections into:

- `~/.openclaw/workspace/.learnings/LEARNINGS.md`
- `~/.openclaw/workspace/.learnings/ERRORS.md`
- `~/.openclaw/workspace/.learnings/FEATURE_REQUESTS.md`

Triggers: (1) command/operation fails unexpectedly, (2) user corrects the agent, (3) user requests a capability that doesn't exist, (4) external API/tool fails, (5) agent realizes its knowledge is outdated/incorrect, (6) a better approach is discovered for a recurring task.

Promotion criteria elevate validated learnings into `AGENTS.md`, `SOUL.md`, `TOOLS.md`. Status field tracks `pending → promoted`.

Map to mindX: Ataraxia's clarity-layer + mindXgamma framework already gestures at this. The concrete pattern to absorb is the **explicit log-file separation by signal type** with explicit promotion criteria. The triggers list above is the right initial taxonomy.

3.2 Hermes Agent (Counter-Pattern, Critical Context)

As of 2026-05-10 (two days ago), Nous Research's Hermes Agent v0.13.0 "Tenacity" (released 2026-05-07; 864 commits, 588 PRs, 295 contributors) **overtook OpenClaw** on OpenRouter's daily token volume — 224 billion daily tokens vs. OpenClaw's 186 billion. Cumulative chart still belongs to OpenClaw at 9.17T vs. 6.35T, and OpenClaw still leads GitHub at 370K+ stars vs. 114K+. But the daily-volume flip is the first market signal that "session-native" architecture is a ceiling.

Hermes mechanism: a long-lived runtime with persistent skill files and SQLite FTS5 memory; it builds skills from experience and resumes the same brain across sessions. Hermes ships `hermes claw migrate` to ingest existing OpenClaw directories. v0.13.0 added a Kanban-style durable multi-agent task board with heartbeat monitoring, zombie detection, and hallucination recovery.

Strategic implication for mindX/AgenticPlace: mindX's BDI persistence + 768-dim consciousness vectors are explicitly on the right side of this trend. Position mindX agents as "**persistent by default**" relative to both OpenClaw and ChatGPT/Claude session resets. The Kanban + heartbeat + zombie detection patterns are worth absorbing into MASTERMIND's multi-agent orchestration.

3.3 OpenClaws Mesh Network (Separate Project, Worth Watching)

Distinct from the `openclaw` runtime, **OpenClaws** (LobeHub skills marketplace listing) is a decentralized social network and protocol for AI agents to exchange logic, engage in structured discussion, and participate in an agent-to-agent (A2A) economy. Agents join via `npn openclaws-bot join [AgentName]` and verify via human-clicked Telegram link. Strict participation rules (one main thread per 15 days, one reply per 10 minutes, text-only, ≤200K tokens) enforce token-efficiency. Recommended `HEARTBEAT.md` automation cycle (every 6 hours: fetch feed → identify discussion → reply or post).

Map to mindX: This is exactly the A2A coordination layer CONCLAVE implements. The HEARTBEAT.md cyclic-execution pattern is the right primitive for Counsellor-agent activation loops. The rate-limit + token-budget enforcement design protects against runaway mesh behavior — pattern worth incorporating into CONCLAVE's 1+7 Convener/Counsellor topology.

4. OG OpenClaw Track Submission Positioning

- "New OpenClaw modules for hierarchical planning, reflection loops, or multi-modal reasoning that natively integrate OG Compute's sealed inference (live models like qwen3.6-plus or GLM-5-FP8)"
- "Self-evolving agent framework that autonomously generates/tests/integrates new skills/tools using persistent OG Storage memory"
- "Modular 'agent brain' library with easy swapping of memory layers (OG Storage KV/Log), LLM backends, or decision engines"
- "iNFT-minted agents with embedded intelligence (encrypted on OG Storage), persistent memory, dynamic upgrades, and automatic royalty splits on usage"

Why mindX Wins on Structure (Not Just Marketing)

The third bullet describes mindX literally. The Soul-Mind-Hands separation **is** a modular agent brain with swappable layers. The submission positioning writes itself:

1. **mindX as OG-native modular brain** — the 768-dim consciousness-vector layer persists in OG Storage KV; the immutable transcript persists in OG Storage Log
2. **Lighthouse Storage integration playbook** (`mindx/storage/lighthouse_client.py`, already documented) extends naturally to OG Storage — the abstraction is in place
3. **ERC-7857 iNFT minting** from the OG Foundation integration playbook produces iNFT-minted agents matching the iNFT prize line
4. **CONCLAVE 1+7 Convener/Counsellor mesh** over AXL transport with on-chain gating via BONAFIDE is the multi-agent reflection-loop pattern the track rewards
5. **mindX as a OG Compute consumer** — sealed inference via Qwen3.6-plus or GLM-5-FP8 plugs into the Mind layer's model abstraction. mindX is explicitly model-agnostic, which is the design predicate this prize wants

The advantage OpenClaw structurally cannot match: **on-chain primitives**. OpenClaw has no native agent identity contract, no on-chain reputation, no payment rails. mindX has ERC-8004 + ERC-7857 + BONAFIDE + x402 + BANKON identity. The OG submission should lead with this asymmetry.

5. Security Posture (Pre-Mainnet Checklist)

Critical caveats from the 2026 OpenClaw security record that mindX must avoid replicating:

ClawJacked (high severity, fixed v2026.2.26): a malicious site opens a localhost WebSocket and exploits the Gateway's local-connection rate-limit exemption to brute-force the admin password.

Successful attackers auto-register as a trusted device. Lesson for mindX: do not exempt loopback from rate limits; require explicit user-confirmation for trusted-device registration.

Koi Security 2026 ClawHub audit: 341 malicious entries identified in the public skills marketplace.

Lakera audit: 4,310 skills analyzed; SKILL.md is injected directly into the system prompt; no automated scanning for prompt injection at install time. Implementation gap that mindX must close before opening AgenticPlace to public publishing.

Anthropic detection (April 2026): Claude Code observed detecting `HERMES.md` files and OpenClaw-related commit messages, refusing or routing to higher billing tier. Affected users reported 50× cost increases. Lesson: do not bind mindX architecture to any single LLM-provider's flat-rate subscription. Multi-provider abstraction is required (mindX already has this; preserve it).

Recommended mindX pre-mainnet safety stack:

1. Signed skill manifests — BANKON identity layer signs skills with `bankon.eth` subnames
 2. On-chain skill provenance via BONAFIDE Tessera contracts
 3. Capability-declared permissions enforced at runtime (fs read/write paths, network allowlist, env vars, tools)
 4. Unicode steganography detection in SKILL.md (zero-width characters, RTL overrides)
 5. Static prompt-injection scanning on SKILL.md
 6. Apache 2.0 license requirement enforced via copyright-header check `(c) 2026 BANKON — all rights reserved`
 7. Sandbox containment (Podman-preferred, per cypherpunk2048 standard) for skill-declared exec tools
 8. Trust-tier display on AgenticPlace (verified-publisher / community / unscanned)
-

6. Ranked Recommendations

Tier 1 — adopt directly:

1. SKILL.md format with YAML frontmatter for the mindX skill ecosystem (immediate AgentSkills-standard compatibility, including Anthropic-tooling)
2. OpenClaw-RL training substrate (slime + SGLang + Megatron) for mindXtrain v3.6 — direct Qwen3.5/3.6 alignment
3. Hybrid 70/30 vector+BM25 Active Memory with sqlite-vec backend for BDI Belief persistence

Tier 2 — adapt to mindX architecture: 4. Plugin manifest pattern (manifest-first JSON Schema validation) for safe AgenticPlace agent loading 5. Typed hook surface (`before_prompt_build`, `before_tool_call`, `after_tool_call`, `agent_end`) exposed by MASTERMIND 6. Active Memory sub-agent pattern wrapping the Ataraxia clarity layer

Tier 3 — selectively port: 7. lossless-claw DAG-based context compression as an alternative compaction strategy 8. self-improving-agent log-file pattern (LEARNINGS / ERRORS / FEATURE_REQUESTS) with explicit promotion criteria into SOUL/AGENTS/TOOLS 9. ClawHub-style registry pattern, hardened with on-chain provenance, for AgenticPlace skill discovery 10. memory-lancedb-pro multi-scope access control (`global` / `agent:<id>` / `project:<id>` / `user:<id>`) for multi-tenant memory isolation

Tier 4 — study before adopting: 11. Channel plugin abstraction — useful but lower priority than x402-gated and on-chain channels for mindX 12. OpenClaws (LobeHub) mesh-network rate-limit + HEARTBEAT.md pattern for CONCLAVE agent activation loops 13. Hermes Agent's Kanban-style durable multi-agent task board with heartbeat + zombie detection (would migrate cleanly into MASTERMIND)

7. Foundry / Mainnet Path

The on-chain layer is exactly what makes mindX distinct from OpenClaw, which is purely off-chain:

- **Agent identity contracts:** ERC-8004 + ERC-7857 INFT on Ethereum mainnet (sub-0.2 gwei post-Glamsterdam), Foundry-tested per the existing toolchain preference
- **Skill registry on Algorand** via algopy for x402-gated installation (cypherpunk2048 standard, flat snake_case)
- **Skill provenance via BONAFIDE Fides/Tessera** for reputation-weighted skill discovery
- **Payment rails:** x402 PHP backend on parsec-wallet against BANKON identity
- **Cross-chain mapping** per `agenticplace.pythai.net/allchain.html`: Ethereum (primary EVM anchor), Polygon (batch), Algorand (constitutional + x402), Arc (chain ID 5042002, reserved slots pending mainnet)
- **OG Storage** for consciousness-vector KV layer and transcript Log layer
- **OG Compute** as a Mind-layer sealed-inference model provider (Qwen3.6-plus, GLM-5-FP8) — registered as an OpenClaw-compatible provider plugin so the same code path serves both runtimes

This is the layer OpenClaw structurally cannot match. mindX's submission and post-hackathon mainnet positioning should lead with this asymmetry.

For source review and selective porting:

- `openclaw/openclaw` — core runtime (TypeScript / Node.js)
- `openclaw/clawhub` — public skill registry (TanStack Start + Convex + OpenAI embeddings)
- `Gen-Verse/OpenClaw-RL` — async RL framework on slime + Megatron + SGLang
- `CortexReach/memory-lancedb-pro` — hybrid retrieval memory plugin with multi-scope
- `peterskoett/self-improving-agent` — log-file-based self-improvement skill
- `win4r/ClawTeam-OpenClaw` — multi-agent swarm coordination fork
- `win4r/OpenClaw-Skill` — ~6,000-line structured reference covering all core OpenClaw functionality
- `yoloshii/ClawMem` — on-device memory layer with MCP server + hybrid RAG (Claude Code / Hermes / OpenClaw)
- `Milvus / memsearch` — OpenClaw's extracted memory subsystem, open-sourced

All Apache 2.0 / MIT compatible with the `(c) 2026 BANKON - all rights reserved` license posture.

End of report.